Reviewer Pack — Minimal Quadratic Form Rank Bound on Frege Proof Depth

Entry 27f15b87a282 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 20:37:57 UTC

Minimal Quadratic Form Rank Bound on Frege Proof Depth

Entry ID: `27f15b87a282` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 20:37:57 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quadratic Forms) **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

> For every CNF φ with n variables, the resolution proof depth d(φ) of φ is lower-bounded by a function f(n) = O(n^(1/2)), and there exists a constant c such that the minimal rank of any quadratic form representing a clause in φ is at least c * log^2(n). Equivalently: d(φ) ≥ Θ(f(n)) and min_{c: φ(c)} rank(Q_c) ≥ c * log^2(n), where Q_c represents a quadratic form associated with clause c.

Rationale (proposer's reasoning):

> Quadratic forms have been studied in algebraic geometry, which may provide insights into the complexity of proving satisfiability. The minimal rank of quadratic forms could potentially relate to the depth of Frege proofs, offering a new perspective on proof complexity. This bridge could expose the structure of Frege proofs and potentially lead to improved algorithms for solving SAT instances.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `25467b6967f4e330`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the correlation coefficient between the minimal rank of the quadratic forms and the Frege proof depth is ≥ 0.8, and no seed produces a correlation coefficient < 0.6.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'algebraic geometry' AND 'quadratic forms' AND 'resolution proof depth' AND 'Boolean satisfiability'` - `'Frege proof complexity' AND 'minimal rank of quadratic forms' AND 'CNF resolution' AND 'proof depth'` - `'log^2(n)' AND 'minimal quadratic form rank' AND 'resolution proof complexity' AND 'algebraic geometry'`

Top relevant hits considered: - [http://arxiv.org/abs/math/0205005v2] The rings of noncommutative projective geometry - [http://arxiv.org/abs/0912.2255v3] Test ideals via algebras of p^{-e}-linear maps - [http://arxiv.org/abs/1812.11710v1] Geometric Satake correspondence for affine Kac-Moody Lie algebras of type A - [http://arxiv.org/abs/1804.02478v5] Complexity of the CNF-satisfiability problem - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def generate_cnf(n):
    cnf = []
    for _ in range(2 * n):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        while any(c == -d for c, d in zip(clause, cnf[-n:])):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        cnf.append(clause)
    return cnf

def calculate_quadratic_form_rank(clause):
    n = len(clause)
    Q = [[0] * n for _ in range(n)]
    for literal in clause:
        var = abs(literal) - 1
        Q[var][var] += 1
    rank = gaussian_elimination(Q)
    return rank

def gaussian_elimination(A):
```

```
    n = len(A)
    for i in range(n):
        if A[i][i] == 0:
            for j in range(i + 1, n):
                if A[j][i] != 0:
                    A[i], A[j] = A[j], A[i]
                    break
            else:
                return 0
        pivot = A[i][i]
        for j in range(n):
            A[i][j] /= pivot
        for j in range(n):
            if j == i:
                continue
            factor = A[j][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return sum(1 for row in A if any(row))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        cnf = generate_cnf(n)
        ranks = []
        depths = []

        for clause in cnf:
            rank = calculate_quadratic_form_rank(clause)
            ranks.append(rank)

            # Simulate Frege proof depth (placeholder, replace with actual calculation)
            depth = random.randint(10, 2 * n)
            depths.append(depth)

        if not ranks or not depths:
            return {
                "metric_name": "quadratic_form_rank",
                "metric_value": None,
                "instances_tested": len(cnf),
                "n_max": max(n_values),
                "conjecture_holds": False,
                "counterexample": "empty_ranks_or_depths"
            }

        correlation = calculate_correlation(ranks, depths)
        results.append({
            "n": n,
            "correlation": correlation
        })

    mean_corr = sum(result["correlation"] for result in results) / len(results)
    min_corr = min(result["correlation"] for result in results)
```

```
    conjecture_holds = all(correlation >= 0.6 for correlation in results)
    if not conjecture_holds:
        first_failing_seed = seed
    else:
        first_failing_seed = None

    return {
        "metric_name": "quadratic_form_rank",
        "metric_value": mean_corr,
        "instances_tested": len(cnf) * len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"correlation < 0.6 at n={min_corr}"
    }

def calculate_correlation(x, y):
    mean_x = sum(x) / len(x)
    mean_y = sum(y) / len(y)
    cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(len(x))) / len(x)
    std_x = math.sqrt(sum((x[i] - mean_x) ** 2 for i in range(len(x))) / len(x))
    std_y = math.sqrt(sum((y[i] - mean_y) ** 2 for i in range(len(y))) / len(y))
    return cov_xy / (std_x * std_y)

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30))  # Generate 30

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_corr = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_corr} std=0.0 support_fraction=1.0")
    elif any(not result["conjecture_holds"] for result in results) and min_corr < 0.6:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='correlation < 0.6' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_100890e7.py", line 123, in <module>
    result = run_trial(seed)
```

```
            ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_100890e7.py", line 63, in run_trial
    cnf = generate_cnf(n)
          ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_100890e7.py", line 22, in generate_cnf
    while any(c == -d for c, d in zip(clause, cnf[-n:])):
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_100890e7.py", line 22, in <genexpr>
    while any(c == -d for c, d in zip(clause, cnf[-n:])):
                   ^^
TypeError: bad operand type for unary -: 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to provide a correlation coefficient. | next: Investigate the cause of the crash and re-run the test with proper error handling to ensure it completes successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15056
2	preregistration	ollama_remote	glm4:latest	0	0	9604
3	novelty	ollama_remote	glm4:latest	0	0	9163
4	novelty	ollama_remote	glm4:latest	0	0	21297
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14416
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10220
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11840
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13780
9	judge	ollama_remote	glm4:latest	0	0	13059

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 118434 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/27f15b87a282.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/27f15b87a282.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/27f15b87a282.tar.gz` (if generated)